

Operating Systems: Assignment-11

Name: Suraj Shah

Admission Number: I24AI012

Q 1. Implementation of resource allocation graph (RAG), where a number of processes, resources exist in the state of a system.

Code:

```
class RAG:
    def __init__(self):
        self.graph={}

    def add_node(self, node):
        if node not in self.graph:
            self.graph[node]=[]

    def request_resource(self, process, resource):
        self.graph[process].append(resource)

    def assign_resource(self, resource, process):
        self.graph[resource].append(process)

    def detect_cycle(self):
        visited=set()
        rec_stack=set()

        def dfs(node):
            visited.add(node)
            rec_stack.add(node)

            for neighbor in self.graph.get(node, []):
                if neighbor not in visited:
                    if dfs(neighbor):
                        return True
                elif neighbor in rec_stack:
                    return True

            rec_stack.remove(node)
            return False

        for node in self.graph:
            if node not in visited:
                if dfs(node):
                    return True
```

```

        return False

rag=RAG()

p=int(input("Enter number of processes: "))
r=int(input("Enter number of resources: "))

print("Enter process names:")
for _ in range(p):
    proc=input()
    rag.add_node(proc)

print("Enter resource names:")
for _ in range(r):
    res=input()
    rag.add_node(res)

e=int(input("Enter number of edges: "))

print("\nEnter edges:")
print("Format:")
print("1 P R -> Process P requests Resource R")
print("2 R P -> Resource R assigned Process P")

for _ in range(e):
    data=input().split()

    if data[0]=='1':
        process=data[1]
        resource=data[2]
        rag.request_resource(process, resource)

    elif data[0]=='2':
        resource=data[1]
        process=data[2]
        rag.assign_resource(resource, process)

if rag.detect_cycle():
    print("\nDeadlock detected!")
else:
    print("\nNo deadlock")

```

Output:

```
Enter number of processes: 2
Enter number of resources: 2
Enter process names:
P1
P2
Enter resource names:
R1
R2
Enter number of edges: 4

Enter edges:
Format:
1 P R -> Process P requests Resource R
2 R P -> Resource R assigned Process P
1 P1 R1
2 R1 P2
1 P2 R2
2 R2 P1

Deadlock detected!
```

Q 2. Consider a state of system where there are number of process, number of resources. A process is requesting resource. Consider a scenario of a resource allocation graph or wait for graph with cycle and without cycle. Implement Cycle Detection Algorithm (DFS-based) in RAG to check for deadlock.

Code:

```
class RAG:
    def __init__(self):
        self.graph={}

    def add_node(self, node):
        if node not in self.graph:
            self.graph[node]=[]

    def add_edge(self, u, v):
        self.graph[u].append(v)

    def is_cyclic_util(self, node, visited, rec_stack):
        visited.add(node)
        rec_stack.add(node)

        for neighbor in self.graph[node]:
            if neighbor not in visited:
                if self.is_cyclic_util(neighbor, visited, rec_stack):
                    return True
            elif neighbor in rec_stack:
                return True

        rec_stack.remove(node)
        return False

    def detect_deadlock(self):
        visited=set()
        rec_stack=set()

        for node in self.graph:
            if node not in visited:
                if self.is_cyclic_util(node, visited, rec_stack):
                    return True
        return False

rag = RAG()

p = int(input("Enter number of processes: "))
r = int(input("Enter number of resources: "))

print("Enter process names:")
for _ in range(p):
```

```

rag.add_node(input())

print("Enter resource names:")
for _ in range(r):
    rag.add_node(input())

e=int(input("Enter number of edges: "))

print("\nEnter edges:")
print("Format:")
print("1 P R  -> Process P requests Resource R")
print("2 R P  -> Resource R assigned to Process P")

for _ in range(e):
    data=input().split()

    if data[0]=='1':
        rag.add_edge(data[1], data[2])
    elif data[0]=='2':
        rag.add_edge(data[1], data[2])

if rag.detect_deadlock():
    print("\nDeadlock detected!")
else:
    print("\nNo deadlock")

```

Output:

```

Enter number of processes: 2
Enter number of resources: 2
Enter process names:
P1
P2
Enter resource names:
R1
R2
Enter number of edges: 4

Enter edges:
Format:
1 P R  -> Process P requests Resource R
2 R P  -> Resource R assigned to Process P
1 P1 R1
2 R1 P2
1 P2 R2
2 R2 P1

Deadlock detected!

```

Q 3. Write a program to implement a way to prevent runtime a possibility of a deadlock occurring in a system described as follows

Write a program with following specifications:

Command line input: name of a file

- The file contains the initial state of the system as given below (example included in right column)

#no of resources 4

#no of instances of each resource 2 4 5 3

#no of processes 3

#no of instances of each resource that 1 1 1 1, 2 3 1 2, 2 2 1 3

each processes needs in its lifetime.

The rule is that once a process uses a resource instance and returns it, the process no more needs it. For example, in the above set of 3 processes, If P2 finishes with 1 instance of resource r3, it will only need $3-1=2$ instances of r3 in future. A process completes immediately when its need eventually reduces to 0.

Read this in

The programs then waits to accept a resource allocation or release requests-this is supplied through keyboard

For example:

0 a 1 0 1 1 indicates that p0 has requested allocation of 1 instance of R0, R2 and R3 each.

or

3 r 0 0 0 3 indicates that p3 is releasing 3 instances of resource r3

Your program should declare the result:

(1) should this request be granted? (i.e. guaranteeing that there will be no deadlock in future considering a worst remaining demand for resources)

(2) if your answer is yes, print the safe sequence in which all remaining needs can be granted one by one and also grant the request, making necessary changes to system's state. If the requesting process's need is NIL, the program internally releases all its resources.

Go back to accept another request till all processes finish with all their needs.

Code:

```
import sys

class Bankers:
    def __init__(self, resources, available, max_need):
        self.n=len(max_need)
        self.m=resources
```

```

        self.available=available[:]
        self.max_need=max_need
        self.allocation=[[0]*self.m for _ in range(self.n)]
        self.need=[row[:] for row in max_need]

    def is_safe(self):
        work=self.available[:]
        finish=[False]*self.n
        safe_seq=[]

        while len(safe_seq)<self.n:
            found=False
            for i in range(self.n):
                if not finish[i] and all(self.need[i][j]<=work[j] for j in
range(self.m)):
                    for j in range(self.m):
                        work[j]+=self.allocation[i][j]
                    finish[i]=True
                    safe_seq.append(i)
                    found=True
            if not found:
                return False, []
        return True, safe_seq

    def request(self, pid, req):
        if any(req[j]>self.need[pid][j] for j in range(self.m)):
            print("Request exceeds process need")
            return

        if any(req[j]>self.available[j] for j in range(self.m)):
            print("Not enough resources available")
            return

        for j in range(self.m):
            self.available[j]-=req[j]
            self.allocation[pid][j]+=req[j]
            self.need[pid][j]-=req[j]

        safe, seq=self.is_safe()

        if safe:
            print("Request granted")
            print("Safe sequence:", ["P"+str(i) for i in seq])

            if all(self.need[pid][j]==0 for j in range(self.m)):
                print(f"Process P{pid} completed, releasing resources")
                for j in range(self.m):
                    self.available[j]+=self.allocation[pid][j]

```

```

        self.allocation[pid][j]=0
    else:
        print("Request denied (unsafe state)")

        for j in range(self.m):
            self.available[j]+=req[j]
            self.allocation[pid][j]-=req[j]
            self.need[pid][j]+=req[j]

    def release(self, pid, rel):
        for j in range(self.m):
            rel[j]=min(rel[j], self.allocation[pid][j])
            self.available[j]+=rel[j]
            self.allocation[pid][j]-=rel[j]
            self.need[pid][j]+=rel[j]

        print("Resources released successfully")

filename=sys.argv[1]

with open(filename, 'r') as f:
    resources=int(f.readline())
    available=list(map(int, f.readline().split()))
    n=int(f.readline())

    max_need=[]
    for _ in range(n):
        max_need.append(list(map(int, f.readline().split())))

banker=Bankers(resources, available, max_need)

print("\nSystem Ready. Enter commands:")

while True:
    cmd=input("\nEnter request (pid a/r values) or 'exit': ")

    if cmd=="exit":
        break

    data=cmd.split()
    pid=int(data[0])
    action=data[1]
    values=list(map(int, data[2:]))

    if action=='a':
        banker.request(pid, values)
    elif action=='r':
        banker.release(pid, values)

```

```
    if all(all(banker.need[i][j]==0 for j in range(banker.m)) for i in
range(banker.n)):
        print("\nAll processes completed. Exiting.")
        break
```

Output:

```
● PS D:\Operating Systems\Assignment-11> python 3.py input.txt

System Ready. Enter commands:

Enter request (pid a/r values) or 'exit': 0 a 1 0 1 1
Request granted
Safe sequence: ['P0', 'P1', 'P2']

Enter request (pid a/r values) or 'exit': exit
```